

Spring Boot — Lecture 4

Dependency Injection (DI) & Inversion of Control (IoC)

1. Why Do We Need Spring Core Concepts?

We could already write clean Java code using OOP principles. So why do we need Dependency Injection and IoC? Because without them, even well-written Java code becomes **tightly coupled** at scale — hard to maintain, extend, and unit test. This lecture demonstrates the problem and its solution using plain Java (no Spring yet).

Spring Core does NOT introduce DI — it automates it. DI is a design principle that exists independently and can be demonstrated with pure Java.

2. The Problem — Tight Coupling

Initial tightly coupled design:

```
// OrderService.java – tightly coupled (BAD)
public class OrderService {
    public void placeOrder() {
        System.out.println("Order placed");
        // Creating its own dependency – this is the problem!
        EmailService notification = new EmailService();
        notification.sendNotification();
    }
}

// EmailService.java
public class EmailService {
    public void sendNotification() {
        System.out.println("Email notification sent");
    }
}
```

What's wrong here?

Problem	Explanation
Tight coupling to concrete class	OrderService directly creates EmailService. To switch to SMS, you must modify OrderService code — violating the Open/Closed Principle.
Single Responsibility broken	OrderService should only handle orders. Creating notification objects is not its job — it is acting as a factory, which it should not be.
Hard to unit test	Testing placeOrder() forces a real email to be sent. You cannot isolate the class.
Not extensible	Every new notification type (SMS, Popup) requires modifying OrderService.

3. Tight vs Loosely Coupled — Real World Analogy

You need to travel Delhi → Chandigarh. If you demand: 'Bus No. 42, driver Ramesh, XYZ company, 9am sharp' — you are tightly coupled. You just need to REACH Chandigarh. The vehicle shouldn't matter. Same idea in code: OrderService just needs A notification sent — it should NOT care if it's Email, SMS, or Popup.

Tightly Coupled	Loosely Coupled
OrderService creates EmailService directly	OrderService uses NotificationService interface
Changing type requires modifying OrderService	Swap type without touching OrderService
Cannot unit test in isolation	Inject FakeEmailService during tests
Breaks SRP + Open/Closed Principle	Both SOLID principles satisfied

4. Step 1 — Code to Interfaces, Not Concrete Classes

```
// NotificationService.java (interface)
public interface NotificationService {
    void sendNotification();
}

// EmailService.java
public class EmailService implements NotificationService {
    @Override
    public void sendNotification() {
        System.out.println("Email notification sent");
    }
}

// SMSService.java
public class SMSService implements NotificationService {
    @Override
    public void sendNotification() {
        System.out.println("SMS notification sent");
    }
}

// PopUpNotificationService.java
public class PopUpNotificationService implements NotificationService {
    @Override
    public void sendNotification() {
        System.out.println("Popup notification sent");
    }
}
```

All three are now interchangeable — they all implement the same interface. A parent interface reference can hold any child object (basic Java polymorphism). BUT: the object is still created inside OrderService — still tightly coupled!

5. Step 2 — Dependency Injection

The fix: don't let OrderService create the dependency. Let someone outside provide it.

Golden Rule of DI:

```
"Don't create your own dependency – get your dependency."
"A class should ask for what it needs, not build everything itself."
```

Constructor Injection (recommended approach):

```
// OrderService.java – after Dependency Injection (GOOD)
public class OrderService {

    private NotificationService notification; // instance variable (interface type)

    // Constructor – accepts the dependency from outside
    public OrderService(NotificationService notification) {
        this.notification = notification;
    }

    public void placeOrder() {
        System.out.println("Order placed");
        notification.sendNotification(); // just USES it, doesn't CREATE it
    }
}
```

Main class wires everything together:

```
// Main.java – acts as the wiring / configuration class
public class Main {
    public static void main(String[] args) {
```

```

// Step 1: Create the dependency
NotificationService notification = new EmailService();

// Step 2: Inject it into OrderService via constructor
OrderService order = new OrderService(notification);

// Step 3: Call business method
order.placeOrder();
// Output:
//   Order placed
//   Email notification sent

// To switch to SMS – change ONLY this one line:
// NotificationService notification = new SMSService();
// OrderService stays completely unchanged!
}

```

To switch to SMS: change only one line in Main. OrderService is never touched. Open/Closed Principle satisfied.

6. Types of Dependency Injection

Type	How it works	Notes
Constructor Injection	Dependency passed as constructor parameter when object is created	Preferred. Dependency required at creation time. Makes field immutable.
Setter Injection	Dependency set via a setter method after object is created	Flexible — can change dependency later. Good for optional dependencies.
Field Injection	Dependency injected directly into a field using @Autowired annotation	Spring only — not possible with plain Java. Covered in later lectures.

Setter Injection example:

```

// Add a setter to OrderService
public void setNotification(NotificationService notification) {
    this.notification = notification;
}

// In Main.java – using setter injection
OrderService order = new OrderService();           // empty constructor
order.setNotification(new EmailService());          // inject via setter
order.placeOrder();

```

7. Benefits of Dependency Injection

Benefit	Why it matters
Loose coupling	OrderService doesn't know which notification type is used. Swap freely.
Open/Closed Principle	OrderService is closed for modification but open for extension via new implementations.
Single Responsibility	OrderService only handles orders. Object creation is someone else's job.
Easier unit testing	Inject a FakeEmailService in tests — no real emails sent, core logic tested cleanly.
Flexibility	Different implementations for different environments (real email in prod, fake in tests).

Unit Testing with FakeEmailService:

```

// FakeEmailService.java – for unit testing only
public class FakeEmailService implements NotificationService {
    @Override

```

```

    public void sendNotification() {
        System.out.println("Dummy email sent"); // no real email triggered!
    }
}

// In test code
OrderService order = new OrderService(new FakeEmailService());
order.placeOrder();
// Tests ONLY the OrderService logic — no external side effects

```

This is the foundation of Mockito mocking in Spring unit tests. DI makes mocking possible — dependencies are injected, not hardcoded.

8. Inversion of Control (IoC)

IoC is what happens as a result of applying DI. The 'control' of creating and managing dependencies is inverted.

Control flow — before vs after DI:

Design	Who creates EmailService?	Control location
Before DI	OrderService creates it: new EmailService() inside placeOrder()	INSIDE OrderService — tightly coupled, hard to change
After DI	Main creates it and passes it into OrderService constructor	OUTSIDE OrderService — control is inverted

IoC = control of object creation moved from INSIDE the class to OUTSIDE. That's literally what 'inversion' means — the direction of control flipped.

IoC vs Dependency Injection — the relationship:

Term	What it is	Role
IoC (Inversion of Control)	A design principle — classes should not control their own dependencies	The WHAT — the goal we want to achieve
Dependency Injection	A technique — pass dependencies from outside rather than creating inside	The HOW — the technique to achieve IoC

9. Problem With Manual DI at Scale — Why We Need Spring

In our example, Main only wires 2 objects. But real applications have hundreds of services all depending on each other. Main becomes a massive wiring class:

```

// Main.java in a real app — gets very complicated very fast
public static void main(String[] args) {
    EmailService email      = new EmailService();
    SMSService sms          = new SMSService();
    UserRepository userRepo = new UserRepository();
    PaymentGateway payment  = new PaymentGateway();
    UserService user        = new UserService(userRepo);
    PaymentService pay      = new PaymentService(payment, user);
    NotifService notif      = new NotifService(email, sms);
    OrderService order      = new OrderService(pay, notif, user);
    // ... 100 more services ...
}

```

This is exactly what the Spring IoC Container solves. It replaces Main as the wirer — automatically.

10. Spring IoC Container — What It Does

The Spring IoC Container takes over everything Main was doing manually.

Job	Before Spring (Main does it)	After Spring (IoC Container does it)
Create objects	<code>new EmailService(); new OrderService(n);</code> — manually	Spring auto-creates all configured beans
Manage lifecycle	Objects live as long as Main holds references	Spring controls creation scope and destruction of beans
Wire dependencies	<code>order.setNotification(n);</code> — manually per class	Spring auto-wires dependencies between beans

In Spring code you will almost never see the 'new' keyword for service objects. Spring creates and manages them all — you just declare what you need.

11. Beans — Spring's Name for Managed Objects

Term	Meaning
Object	Any Java instance created with 'new'. Not managed by Spring. Lives and dies normally.
Bean	A Java object whose lifecycle is managed by the Spring IoC Container. Spring creates it, wires its dependencies, and destroys it when done.

- Every Bean is an Object — but not every Object is a Bean.
- Only objects managed by the Spring IoC Container are Beans.
- Spring creates Beans, wires them together, and manages their full lifecycle.
- We will work with Beans using `@Bean`, `@Component`, `@Autowired` in upcoming lectures.

12. Project Classes Built in This Lecture

Class / Interface	Type	Role
NotificationService	Interface	Contract for all notification types. Method: <code>sendNotification()</code>
EmailService	Class → implements NotificationService	Sends email. Prints 'Email notification sent'.
SMSService	Class → implements NotificationService	Sends SMS. Prints 'SMS notification sent'.
PopUpNotificationService	Class → implements NotificationService	Sends popup. Prints 'Popup notification sent'.
FakeEmailService	Class → implements NotificationService	Dummy for testing. Prints 'Dummy email sent'. No real email triggered.
OrderService	Business logic class	Places orders. Accepts NotificationService via constructor — does not create it.
Main	Entry point / wiring class	Creates objects and injects dependencies manually (replaced by IoC Container in Spring).

Package structure:

```
in.coderarmyarmy.coredemo/
  notification/
    NotificationService.java      (interface)
    EmailService.java
    SMSService.java
    PopUpNotificationService.java
    FakeEmailService.java        (for testing)
    OrderService.java
```

Quick Reference — Key Terms

Term	One-line Definition
Tight Coupling	A class creates its own dependencies — hard to change, test, or extend.
Loose Coupling	A class uses dependencies without knowing how they are created.
Dependency	An object a class needs to do its work.
Dependency Injection	Passing dependencies to a class from outside rather than letting it create them.
Constructor Injection	Dependency passed as constructor parameter at object creation time.
Setter Injection	Dependency set via a setter method after the object is created.
Field Injection	Dependency injected directly into a field — Spring only (@Autowired).
IoC	Inversion of Control — control of object creation moved outside the class.
IoC Container	Spring's engine that creates, manages, and wires all Bean objects.
Bean	A Java object whose lifecycle is fully managed by the Spring IoC Container.
SRP	Single Responsibility Principle — a class should have one reason to change.
OCP	Open/Closed Principle — open for extension, closed for modification.

Next Lecture: Spring IoC Container in action — @Component, @Autowired, Beans with Spring